UNIVERSIDAD MARIANO GÁLVEZ DE GUATEMALA

AutoAudit

Sistema de auditoría automatizada de bases de datos relacionales

ENTORNO	TECNOLOGÍAS	ENTREGA
Ubuntu 26.04 LTS	PostgreSQL 18 · PL/pgSQL · Python · Flask	Documentación + código + pruebas

Curso: Bases de Datos I **Facultad:** Ingeniería en Sistemas de Información y Ciencias de la Computación

Estudiante: Hector Daniel Escobar Gramajo

Carné: 2590-22-22530 **Docente:** René Ornélyz

Proyecto de Bases de Datos I. Configuración de auditorías de tablas relacionales, registro histórico de cambios y consultas por usuario y columna.

01 Descripción, objetivos y alcance

AutoAudit es un sistema configurable que identifica tablas y columnas de PostgreSQL a través del diccionario de datos, genera triggers automáticamente y conserva un registro histórico centralizado de operaciones DML.

Objetivo principal

Capturar modificaciones de tablas configurables usando triggers dinámicos, con consultas históricas por usuario y por columna, conservando el valor anterior y nuevo de cada cambio observado.

Componentes exigidos por el proyecto

MÓDULO	FUNCIONALIDAD IMPLEMENTADA
Configuración	Selección de tabla, columna y operación; activación/desactivación; reglas persistidas.
Generación de triggers	Creación y regeneración automática de triggers AFTER ROW por tabla y operación.
Bitácora	Fila completa antes/después en JSONB, detalle de columnas, fecha, usuario de BD e IP.
Consulta y reporte	Filtros por usuario, columna, operación, fecha; vista cronológica y exportación CSV.

Criterios de diseño

Se preserva el historial cuando la fila de origen desaparece. La configuración nunca se elimina físicamente al deshabilitarla. Los identificadores de tablas y columnas se validan contra el catálogo PostgreSQL y se entrecomillan al generar DDL.

Aclaración de compatibilidad: aunque el enunciado menciona PL/SQL, PostgreSQL utiliza **PL/pgSQL**; el código se desarrolla con el lenguaje correcto del motor requerido.

02 Arquitectura de la solución

CAPA	ARTEFACTO	RESPONSABILIDAD
Presentación	app/main.py y plantillas HTML/CSS	Formulario administrativo, filtros de consulta, visualización y CSV.
Lógica de reglas	autoaudit.configurar_tabla()	Validación de catálogo, persistencia de selección y regeneración.
DDL dinámico	autoaudit.regenerar_triggers()	Eliminación y creación segura de triggers AFTER para cada tabla.
Captura transaccional	autoaudit.capturar_cambio()	Captura row OLD/NEW, usuario, IP y diferencias por columna.
Persistencia	8 tablas del esquema autoaudit	Configuraciones, usuarios, estaciones, eventos y bitácoras.
Exposición histórica	autoaudit.v_historial	Consulta cronológica por tabla, usuario y período.

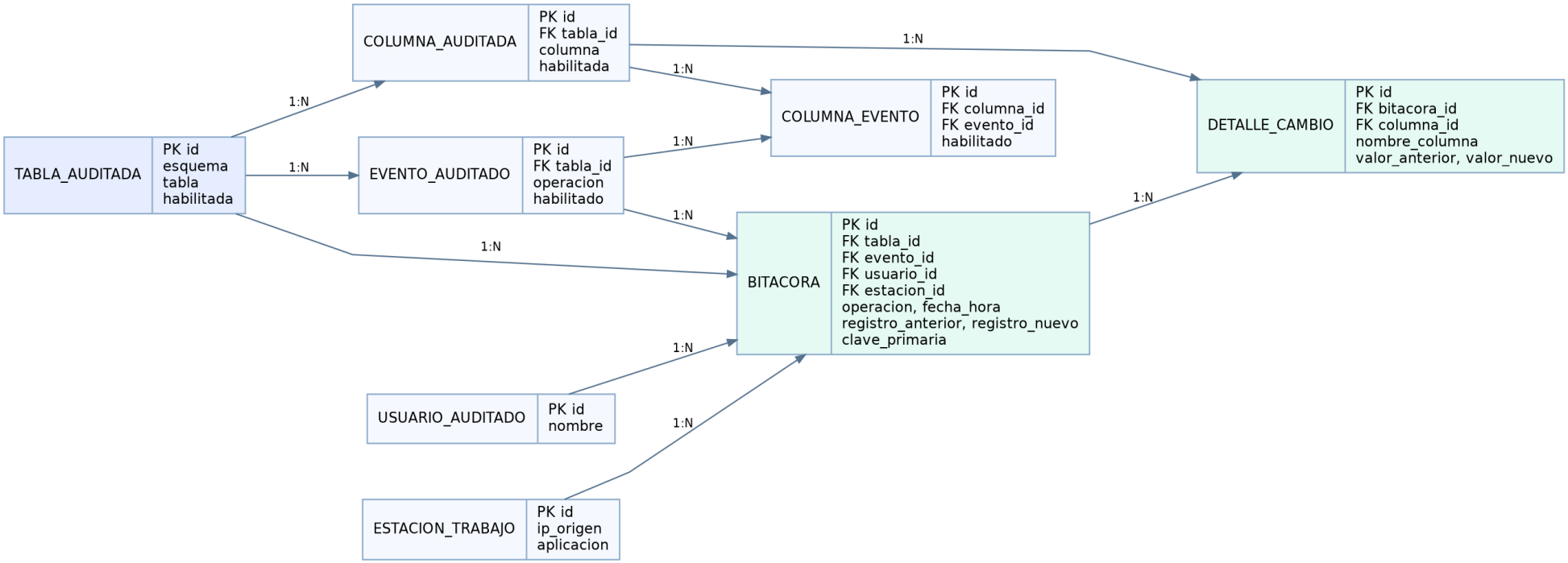
Secuencia de operación

1. El administrador elige columnas y operaciones. **2.** El motor valida la tabla y guarda la configuración. **3.** Se crean triggers específicos para INSERT, UPDATE o DELETE. **4.** Cuando cambia una fila, el trigger determina qué columnas configuradas cambiaron. **5.** Se guarda el encabezado histórico y sus detalles en la misma transacción. **6.** La interfaz muestra y exporta el resultado.

Alcance preciso

Solo se configuran tablas ordinarias no particionadas. El sistema no audita TRUNCATE, DDL, lecturas SELECT ni transacciones que hayan sido revertidas. La identidad registrada es la cuenta de conexión PostgreSQL (`session_user`).

03 Diagrama entidad-relación



Ocho entidades conectadas con relaciones 1:N. La relación N:M entre columnas y operaciones se resuelve mediante la tabla puente COLUMNA_EVENTO.

Regla de conservación histórica

BITACORA mantiene una clave foránea hacia TABLA_AUDITADA (configuración), no hacia la fila física de la tabla origen. Una FK al origen impediría conservar el historial después de una eliminación, por lo que se almacena la clave primaria y las imágenes anterior/nueva de la fila en JSONB.

04 Diccionario de datos (parte I)

tabla_auditada

CAMPO	TIPO	FUNCIÓN / RESTRICCIÓN
id	BIGINT · PK	Identificador estable; IDENTITY
esquema	TEXT	Esquema de la tabla; clave única con tabla
tabla	TEXT	Nombre de tabla origen; único con esquema
habilitada	BOOLEAN	Activación general
creada_en / actualizada_en	TIMESTAMPTZ	Trazabilidad de configuración

columna_auditada

CAMPO	TIPO	FUNCIÓN / RESTRICCIÓN
id	BIGINT · PK	Identificador IDENTITY
tabla_id	BIGINT · FK	Referencia a tabla_auditada, ON DELETE RESTRICT
columna	TEXT	Nombre del campo; único por tabla
habilitada	BOOLEAN	Selección actual

evento_auditado

CAMPO	TIPO	FUNCIÓN / RESTRICCIÓN
id	BIGINT · PK	Identificador IDENTITY
tabla_id	BIGINT · FK	Referencia a tabla_auditada
operacion	TEXT	CHECK: INSERT / UPDATE / DELETE; único por tabla
habilitado	BOOLEAN	Estado de operación de tabla

04 Diccionario de datos (parte II)

columna_evento

CAMPO	TIPO	FUNCIÓN / RESTRICCIÓN
id	BIGINT · PK	Identificador IDENTITY
columna_id	BIGINT · FK	Referencia a columna_auditada
evento_id	BIGINT · FK	Referencia a evento_auditado
habilitado	BOOLEAN	Regla de auditoría específica por columna y evento

usuario_auditado y estacion_trabajo

TABLA	CAMPO	TIPO	DESCRIPCIÓN
usuario_auditado	id / nombre	BIGINT / TEXT	Actor de BD; nombre UNIQUE obtenido de session_user
usuario_auditado	creado_en	TIMESTAMPTZ	Fecha de primera aparición
estacion_trabajo	id	BIGINT · PK	Identificador de estación
estacion_trabajo	ip_origen	INET	IP de cliente; puede ser NULL para socket local
estacion_trabajo	aplicacion	TEXT	Nombre de aplicación de conexión

04 Diccionario de datos (parte III)

bitacora

CAMPO	TIPO	DESCRIPCIÓN
id	BIGINT · PK	Identificador de evento
tabla_id / evento_id	BIGINT · FK	Tabla y operación configuradas
usuario_id / estacion_id	BIGINT · FK	Identidad y origen de la conexión
operacion / fecha_hora	TEXT / TIMESTAMPTZ	Operación DML, instante de captura
transaccion_id / clave_primaria	BIGINT / JSONB	Transacción y valores de la PK de origen
registro_anterior / registro_nuevo	JSONB	Fila completa en versiones anterior y nueva

detalle_cambio

CAMPO	TIPO	DESCRIPCIÓN
id / bitacora_id	BIGINT · PK / FK	Detalle identificado por evento
columna_id / nombre_columna	BIGINT · FK / TEXT	Configuración y nombre conservado
valor_anterior / valor_nuevo	JSONB	Valores de columna anteriores y nuevos

05 Seguridad, integridad y restricciones

Control administrativo

- Solo los miembros del rol **autoaudit_admin** tienen permiso de ejecutar las funciones de configuración.
- Se revoca a PUBLIC el acceso al esquema, tablas, secuencias y funciones de auditoría.
- La interfaz valida HTTP Basic y un token antifalsificación (CSRF), con secreto y claves almacenados en .env.
- Los triggers utilizan **SECURITY DEFINER** para escribir sin otorgar INSERT directo sobre la bitácora.

Integridad y privacidad

- Claves foráneas entre configuración, bitácora, usuarios, estaciones y detalles; el histórico no se borra al desactivar.
- Captura de IP mediante inet_client_addr(); en conexiones locales por socket la IP puede ser NULL.
- Registros completos pueden contener información personal. Definir quién puede consultarlos y cuánto tiempo se retienen.
- Los cambios se auditan en la misma transacción: un ROLLBACK revierte también su evento histórico.

Limitaciones y mitigaciones

RIESGO	MITIGACIÓN / ALCANCE
Rendimiento por fila	Índices en fecha, usuario y columna; medir en 100/1 000/5 000 filas.
Nombres de tabla no confiables	Validación en pg_catalog, quoteo mediante format(%I).
Borrado de una fila origen	Imágenes de fila guardadas en JSONB sin FK a dicha fila.
Cuenta de BD compartida	Session_user identifica la cuenta de conexión, no necesariamente al usuario humano.
Acceso externo a Flask	Entorno de desarrollo enlazado a localhost; HTTPS + WSGI para producción.

06 Instalación, despliegue y activación

Prerequisitos: Ubuntu 26.04 LTS, PostgreSQL 18, psql y Python 3.12 o posterior. La instalación de servidor PostgreSQL depende de la configuración de paquetes del equipo.

Paso 1. Crear la base de datos e instalar el esquema

```
sudo -u postgres createdb autoaudit_db
sudo -u postgres psql -d autoaudit_db -f sql/01_schema.sql
sudo -u postgres psql -d autoaudit_db -f sql/02_functions.sql
sudo -u postgres psql -d autoaudit_db -f sql/03_security.sql
```

Paso 2. Cuenta administrativa de aplicación

```
CREATE ROLE aa_ui LOGIN PASSWORD 'REEMPLAZAR_CLAVE_SEGURA';
GRANT CONNECT ON DATABASE autoaudit_db TO aa_ui;
GRANT autoaudit_admin TO aa_ui;
```

Paso 3. Interfaz web local

```
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
cp .env.example .env  # editar credenciales, secreto y conexión
python run.py
```

Paso 4. Configurar una tabla

Abrir http://127.0.0.1:5000, autenticarse y escoger una tabla. Seleccionar columnas y operaciones INSERT/UPDATE/DELETE, guardar y comprobar en pg_trigger que se crearon los triggers AFTER apropiados.

Nota sobre permisos

El propietario de las funciones que crean triggers debe tener permisos TRIGGER sobre las tablas de negocio. La instalación docente como administrador de PostgreSQL satisface esta condición.

07 Ejemplo SQL y consultas históricas

Configuración y operaciones de muestra

```
CREATE SCHEMA IF NOT EXISTS demo;
CREATE TABLE demo.clientes (id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  nombre TEXT NOT NULL, saldo NUMERIC(12,2));
SELECT autoaudit.configurar_tabla('demo','clientes',
 '{"nombre":["INSERT","UPDATE","DELETE"],"saldo":["UPDATE","DELETE"]}'::jsonb,
 '["INSERT","UPDATE","DELETE"]'::jsonb, TRUE);
INSERT INTO demo.clientes(nombre, saldo) VALUES ('Ana', 100);
UPDATE demo.clientes SET saldo = 125 WHERE nombre = 'Ana';
DELETE FROM demo.clientes WHERE nombre = 'Ana';
```

Consulta cronológica general y por usuario

```
SELECT * FROM autoaudit.v_historial
ORDER BY fecha_hora DESC LIMIT 100;

SELECT * FROM autoaudit.v_historial
WHERE usuario = 'postgres' ORDER BY fecha_hora DESC;
```

Consulta por columna afectada

```
SELECT v.fecha_hora, v.usuario, v.operacion, d.nombre_columna,
  d.valor_anterior, d.valor_nuevo
FROM autoaudit.v_historial v JOIN autoaudit.detalle_cambio d
  ON d.bitacora_id=v.id
WHERE d.nombre_columna='saldo' ORDER BY v.fecha_hora DESC;
```

El directorio sql incluye consultas al diccionario de datos y el ejemplo ejecutable 05_demo.sql. La interfaz presenta filtros equivalentes y permite descargar un archivo CSV UTF-8.

08 Pruebas funcionales de aceptación

CASO	OPERACIÓN	RESULTADO ESPERADO
P01	INSERT con nombre y saldo	Bitácora INSERT, registro_nuevo completo y detalle por columna.
P02	UPDATE de columna no seleccionada	Sin registro de auditoría.
P03	UPDATE de saldo (10 → 20)	Valor anterior y nuevo capturados; solo columna afectada.
P04	DELETE del registro	registro_anterior conservado, registro_nuevo NULL.
P05	Desactivar auditoría e insertar	Sin nuevo evento; se preserva historial previo.
P06	Reactivar UPDATE saldo	Nuevo evento con reglas actualizadas.
P07	Configurar columna inexistente	La función lanza error; no se modifica configuración.
P08	Filtrar por usuario y columna	Solo modificaciones coincidentes en orden cronológico.
P09	Exportar CSV filtrado	Archivo descargable con encabezados y valores.
P10	Usuario no administrativo	No puede configurar ni escribir directamente en bitácora.

Scripts incluidos

Los casos P01–P06 se encuentran definidos mediante aserciones PL/pgSQL en **tests/01_pruebas_funcionales.sql**. El documento **docs/PLAN_PRUEBAS.md** recoge los procedimientos y los resultados esperados para los casos P07–P10.

09 Pruebas de rendimiento y cronograma

El análisis de rendimiento compara las operaciones realizadas sobre una tabla con auditoría frente a una tabla equivalente sin auditoría, mediante consultas SQL instrumentadas.

Metodología

El archivo **tests/02_rendimiento.sql** contiene escenarios de inserción masiva para tres lotes. El análisis utiliza **EXPLAIN (ANALYZE, BUFFERS)**, con registros equivalentes en ambos escenarios.

LOTE	ESCENARIO DE REFERENCIA	ESCENARIO AUDITADO	INDICADORES
100 filas	INSERT sin auditoría	INSERT con triggers	Duración y buffers
1 000 filas	INSERT sin auditoría	INSERT con triggers	Duración y buffers
5 000 filas	INSERT sin auditoría	INSERT con triggers	Duración y buffers

Cálculo de sobrecarga: (tiempo_con_auditoría - tiempo_sin_auditoría) / tiempo_sin_auditoría × 100.

Cronograma tentativo de la especificación

FASE	ACTIVIDADES	REFERENCIA DE CALENDARIO
Diseño	Requisitos, modelo ER, diseño interfaz	Septiembre 2026
Implementación	DDL, diccionario, configuración, generación, reportes	Septiembre - octubre 2026
Pruebas y cierre	Unitarias, integración, ajustes, documentación	Finales de octubre - noviembre 2026
Entrega final	Documentación, código y validación	3 de noviembre de 2026, según diagrama adjunto

10 Lista de entregables y criterios de aceptación

REQUISITO DEL DOCUMENTO	UBICACIÓN EN EL PAQUETE
Diagrama entidad-relación completo	docs/DIAGRAMA_ER.png y DIAGRAMA_ER.dot
Diccionario de datos detallado	docs/DICCIONARIO_DATOS.md y este PDF
Manual de configuración por tabla	docs/MANUAL_CONFIGURACION.md
Instalación/despliegue de triggers	README.md + scripts/instalar.sh
Consultas del catálogo DBMS	sql/04_consultas.sql
Creación de tablas de auditoría	sql/01_schema.sql
Generación dinámica de triggers	sql/02_functions.sql
Consultas parametrizadas por usuario/columna	app/main.py + sql/04_consultas.sql
Ejemplos de uso y casos funcionales	sql/05_demo.sql + tests/01_pruebas_funcionales.sql
Pruebas de rendimiento	tests/02_rendimiento.sql
Interfaz de configuración y reportes	app/main.py + app/templates/ + app/static/
Repositorio de código fuente	Estructura de carpetas sql/, app/, scripts/ y tests/

Organización del proyecto

Los archivos SQL se encuentran en **sql/**, la interfaz en **app/**, los scripts de instalación en **scripts/** y los casos de prueba en **tests/**. Esta documentación corresponde al directorio **docs/**.

Fuente: «AutoAudit - Sistema de auditoría automatizada», proyecto del curso, Facultad de Ingeniería en Sistemas de Información y Ciencias de la Computación, agosto 2026.